

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো
(Prof. Chester Rebeiro)
কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering)
ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাস
(Indian Institute of Technology, Madras)
সপ্তাহ – 05
বক্তৃতা – 21
লিনাক্সে শিডিউলিং
(Scheduling in Linux)(O(n) এবং O(1) Scheduler)

হ্যালো। এই বক্তৃতাটিতে আমরা লিনাক্স অপারেটিং সিস্টেমগুলির scheduling নিয়ে আলোচনা করব। মূলত, আমরা দেখব কিভাবে লিনাক্স কার্নেলের scheduler গুলি সময়ের সাথে প্রবর্তিত হয়েছে। এই বক্তৃতাটির জন্য রেফারেন্স বই, Understanding the Linux Kernel, Third Edition by Daniel Bovet and Marco Cesati দেখুন।

(স্লাইড সময় : 00:38 দেখুন)

লিনাক্স সিস্টেম প্রসেসগুলিকে 2 টি শ্রেণীতে বিভক্ত করে, একটি রিয়েল টাইম প্রসেস (Real Time Process) এবং অন্যটি হল স্বাভাবিক প্রসেস (Normal Process)। মূলত একটি প্রসেস হয় Real Time অথবা স্বাভাবিক প্রসেস হতে পারে। Real Time Process গুলির কঠোর সময়সীমা থাকে। যেমন, রোবট নিয়ন্ত্রণকারী বা ডেটা অর্জন (ডেটা একুইজিশন সিস্টেম, data acquisition system) বা EIC কন্ট্রোলার নিয়ন্ত্রণের সাথে জড়িত প্রসেসগুলি। রিয়েল টাইম প্রসেসগুলি নির্দিষ্ট সময়সীমা লঙ্ঘন করতে পারে না এবং এগুলিকে কোন কম অগ্রাধিকার (low priority) টাস্ক দ্বারা অবরুদ্ধ (block) করা যায় না। অন্য প্রকারের প্রসেসগুলি হল নন রিয়েল টাইম প্রসেস অথবা নর্মাল প্রসেস। নর্মাল প্রসেসগুলি দুই ধরনের হয় যথা, ইন্টারেক্টিভ (interactive) এবং ব্যাচ (batch)। একটি ইন্টারেক্টিভ প্রসেস ক্রমাগত ব্যবহারকারীদের (user) সাথে কথোপকথন (interaction) করতে থাকে। এই

কারণে কীবোর্ড (keyboard) ও মাউস (mouse) অপারেশনের জন্য অপেক্ষা করতে অনেক সময় ব্যয় হয়। এই প্রসেসগুলিকে I/O বাউন্ড প্রসেস বলা হয়। একটি ইনপুট পাওয়া হলে, সাধারণত প্রসেসটির 50 থেকে 150 মিলিসেকেন্ডের মধ্যে জেগে ওঠা উচিত। যদি এটি হতে 150 মিলিসেকেন্ডের বেশি বিলম্ব হয় তাহলে সিস্টেমটিকে মন্থর মনে হবে এবং user সিস্টেম ব্যবহার করতে খুব স্বচ্ছন্দ বোধ করবেন না। অন্যদিকে, ব্যাচ প্রসেসগুলিকে CPU- বাউন্ড প্রসেস বলা যেতে পারে এবং এগুলির খুব বেশি ইউজার ইন্টারাকশন (user interaction) এর প্রয়োজন হয় না। এই প্রসেসগুলি সাধারণত ব্যাকগ্রাউন্ডে চলে। যেমন – gcc compilation বা MATLAB এর মতো কিছু বৈজ্ঞানিক অপারেশন এই ধরনের প্রসেসগুলির মধ্যে পড়ে।

(স্লাইড সময় :02:15 দেখুন)

একবার একটি প্রসেসকে real time প্রসেস হিসেবে চিহ্নিত করা হলে সেটিকে সর্বদা real time প্রসেস হিসেবেই বিবেচনা করা হয়। অর্থাৎ user একটি প্রসেসকে real time হিসেবে নির্দিষ্ট করলে, সেটিকে real time প্রসেস হিসেবেই বিবেচনা করা হবে, এমনকি যদি প্রসেসটি ঘুমানো ছাড়া কিছুই না করে তবুও। (স্লাইড সময় :02:38 দেখুন)

অন্যদিকে, যে প্রসেসগুলি non real time বা নর্মাল প্রসেস সেগুলি কখনো interactive প্রসেস আবার কখনো batch প্রসেস হিসেবে আচরণ করতে পারে। মূলত একটি নর্মাল প্রসেস একটি interactive প্রসেস বা একটি ব্যাচ প্রসেস হিসাবে আচরণ করতে পারে এবং এই আচরণ সময় সময় পরিবর্তিত হতে পারে। সুতরাং, interactive এবং ব্যাচ প্রসেসের মধ্যে পার্থক্য করার জন্য, লিনাক্স অভ্যুদ্যুতনিক হিউরিস্টিকস (heuristics) ব্যবহার করে এবং প্রক্রিয়াটির অতীত আচরণের উপর ভিত্তি করে সিদ্ধান্ত নেয় যে কোনও প্রসেস ব্যাচ বা interactive প্রসেস হিসাবে বিবেচিত হবে কিনা। সুতরাং, আমরা এই সম্পর্কে আরও বিস্তারিত আলোচনা করব।

(স্লাইড সময় :03:22 দেখুন)

সুতরাং, বেশ কয়েক বছরের ইতিহাস দেখলে দেখা যাবে যে লিনাক্স 2.4 থেকে 2.6 এ একটি scheduler ব্যবহার করা হয়েছিল যাকে $O(n)$ scheduler বলা হত। এরপর লিনাক্স 2.6 থেকে 2.6.22 পর্যন্ত scheduler কে $O(1)$ scheduler বলা হত এবং সাম্প্রতিক শিডিউলারটিকে CFS শিডিউলার বলা হয়। পরবর্তী অংশে আমরা এই শিডিউলারগুলি নিয়ে আলোচনা করব।

(স্লাইড সময় :03:54 দেখুন)

$O(n)$ শিডিউলার দিয়ে শুরু করা যাক। শিডিউলারটি সর্বাধিক n দৈর্ঘ্য বিশিষ্ট রেডি প্রসেসের কিউ (ready queue) পরীক্ষা করে যে প্রসেসটি এক্সিকিউট করা প্রয়োজন সেটিকে নির্বাচন করে। এই কারণে এটিকে $O(n)$ শিডিউলার বলা হয়। সুতরাং, মূলত, প্রতিবার context switch এর সময় $O(n)$ শিডিউলারটি runnable প্রসেসগুলির তালিকাটি পরীক্ষা করে, প্রসেসগুলির অগ্রাধিকার (priority) বিচার করে এবং সবচেয়ে উপযুক্ত প্রসেসটিকে running এর জন্য নির্বাচন করে। যেহেতু আমরা ধরেছি যে ready queue তে সর্বাধিক n টি প্রসেস থাকতে পারে, runnable প্রসেসগুলির তালিকাটি পরীক্ষা করা একটি n -order কাজ। স্বভাবতই এটি scalable নয়। স্কেলেবল নয় কারণ যেহেতু n যত বৃদ্ধি পায় অর্থাৎ রেডি কিউতে আর বেশি করে প্রসেস যুক্ত হয় ফলে context switch করার সময়ও তত বৃদ্ধি পায়। অপরিহার্যভাবে, context switch এর overhead বৃদ্ধি পায় যা কখনোই কাম্য নয়। ২০০০ সাল নাগাদ জাভা (java) সূচনা হওয়ার পর $O(n)$ শিডিউলারের এই scalability সমস্যা পরিলক্ষিত হয়। বিশেষ করে Java Virtual Machine (JVM) একাধিক কার্য বা টাস্ক (task) তৈরী করে যার প্রতিটি ready queue তে অধিষ্ঠান করে। সুতরাং শিডিউলারকে এই ready queue পরীক্ষা করে প্রসেস নির্বাচন করতে হয়। $O(n)$ শিডিউলারের অপর একটি সীমাবদ্ধতা হল এই যে এটি SMP সিস্টেমের জন্য কেবলমাত্র একটি সাধারণ (global) কিউ ব্যবহার কর হয়। আমরা আগের বক্তৃতায় দেখেছি যে মাল্টিপ্রসেসর সিস্টেমে একটি গ্লোবাল কিউ ব্যবহার করায় সেটি scalable নয়।

(স্লাইড সময় :05:48 দেখুন)

সুতরাং পরবর্তী বিকল্পটি হল যাকে বলা হয় $O(1)$ শিডিউলার। রেডি কিউতে যে কয়টা প্রসেসই থাকুক না কেন $O(1)$ শিডিউলার একটি ধ্রুবক সময়ের (constant time) মধ্যে নির্বাচনের সিদ্ধান্ত নিতে পারে। অর্থাৎ পরবর্তী কোন প্রসেস CPU তে এক্সিকিউট করবে তাঁর সিদ্ধান্ত নির্দিষ্ট সময়ের মধ্যেই নেওয়া হয়। স্বভাবতই রেডি কিউতে অনেক প্রসেস থাকলেও এই পদ্ধতিতে scalability বজায় থাকে। যেমন আমরা দেখেছি, $O(1)$ শিডিউলারে প্রসেসগুলিকে দুইটি ধরণে ভাগ করা হয় যথা real time এবং normal প্রসেস। Real time প্রসেসগুলিকে 0 থেকে 99 অগ্রাধিকার সংখ্যা (priority number) দেওয়া হয়, 0 মানে সর্বোচ্চ অগ্রাধিকার (priority) এবং 99 মানে সর্বনিম্ন অগ্রাধিকার। Normal প্রসেসগুলিকে 100 থেকে 139 অগ্রাধিকার সংখ্যা দেওয়া হয়, যেখানে 100 মানে প্রসেসের সর্বোচ্চ অগ্রাধিকার এবং 139 মানে প্রসেসের সর্বনিম্ন অগ্রাধিকার। পূর্বে আমরা দেখেছি যে normal প্রসেসগুলি interactive বা batch হতে পারে। পরে আমরা দেখব শিডিউলার কিভাবে interactive এবং batch প্রসেসের মধ্যে প্রভেদ করতে পারে।

(স্লাইড সময় :07:00 দেখুন)

এবার আমরা normal অর্থাৎ যেগুলি real time প্রসেস নয় সেগুলি নিয়ে আলোচনা করব। মূলত, এখানে যে শিডিউলিং অ্যালগরিদমটি প্রয়োগ করা হয় সেটি multi level feedback queue scheduling এর মতই, যদিও কিছুটা পার্থক্য আছে। এই শিডিউলারে 40 টি অগ্রাধিকার শ্রেণী (priority class) থাকে, যেগুলিকে 100 থেকে 139 ক্রমিক সংখ্যা দেওয়া হয়। যেমন আগেই বলা হয়েছে, সর্বনিম্ন অগ্রাধিকারটি হল 139 এবং সর্বোচ্চ অগ্রাধিকার হল 100। প্রতিটি অগ্রাধিকার শ্রেণী (priority class) এর টাস্কের জন্য আলাদা রেডি কিউ থাকে। অর্থাৎ একটি অগ্রাধিকার শ্রেণীর রেডি কিউতে থাকা প্রতিটি প্রসেসের CPU তে এক্সিকিউট করার অগ্রাধিকার সমান। অপর দিকে 100 তম অগ্রাধিকার শ্রেণীতে যে প্রসেস আছে সেটির অগ্রাধিকার 102 অগ্রাধিকার শ্রেণীতে যে প্রসেস আছে তার অপেক্ষা বেশি। এই অগ্রাধিকার শ্রেণী অনুযায়ী শিডিউলারটি দুইটি কিউ বজায় রাখে। একটিকে এক্টিভ রান কিউ (Active Run queue) এবং অপরটিকে এক্সপায়ারড রান কিউ (Expired Run queue) বলা হয়।

(স্লাইড সময় :08:26 দেখুন)

যখন একটি context switch হয়, তখন শিডিউলারটি 100 তম থেকে শুরু করে 139 তম এক্টিভ রান কিউগুলিকে পরীক্ষা করতে থাকে। যখনই খালি নয় এমন একটি কিউ পাওয়া যায় তখনই সেখান থেকে একটি প্রসেসকে CPU তে এক্সিকিউট করতে পাঠানো হয়। ধরি এটা 100 থেকে শুরু হচ্ছে আর প্রথম প্রসেসটিকে এক্সিকিউট করতে CPU তে পাঠায়। নির্দিষ্ট সময় (time slice) শেষ হলে প্রসেসটিকে এক্সপায়ারড রান কিউ পাঠানো হয়। সুতরাং দেখা যাচ্ছে যে সময়ের সাথে এবং time slice শেষ হলে শিডিউলার প্রসেসগুলিকে এক্টিভ রান কিউ থেকে নেয়, CPU দ্বারা এক্সিকিউট করে এবং এক্সপায়ারড রান কিউতে পাঠায়। কিছুক্ষণ পরে এমন একটি সময় হবে যখন এক্টিভ রান কিউতে আর কোন প্রসেস থাকবে না। অর্থাৎ এক সময় হবে যে কোন অগ্রাধিকার শ্রেণী (priority class) তেই কোন প্রসেস পড়ে থাকবে না। অপরদিকে এক্সপায়ারড রান কিউটি ভর্তি হয়ে যাবে যেহেতু শিডিউলারটি এক্সপায়ারড রান কিউইয়ের প্রতিটি অগ্রাধিকার শ্রেণী (priority class) তে প্রসেস যোগ করতে থাকবে।

(স্লাইড সময় :09:36 দেখুন)

এবার শিডিউলার এক্সপায়ারড রান কিউগুলি এবং এক্টিভ রান কিউগুলি বিনিময় করে দেবে। অর্থাৎ, পূর্ববর্তী এক্সপায়ারড রান কিউগুলিকে এক্টিভ রান কিউ হিসেবে ধরা হবে এবং পূর্ববর্তী এক্টিভ রান কিউগুলিকে এক্সপায়ারড রান কিউ হিসেবে ধরা হবে। এবার উপর্যুপরি time slice সম্পূর্ণ যেমন যেমন হবে পরিবর্তিত এক্টিভ রান কিউ থেকে প্রসেস নিয়ে CPU তে এক্সিকিউট করে পরিবর্তিত এক্সপায়ারড রান কিউতে রাখা হয়। এভাবে বার বার এক্সপায়ারড রান কিউগুলি এবং এক্টিভ রান কিউগুলি বিনিময় করে প্রসেসগুলিকে এক্সিকিউট

করা হতে থাকে। লিনাক্স কার্নেল শিডিউলারে এই পদ্ধতি অনুসরণ করার কারণ হল এই যে এর ফলে কোন প্রসেসের অনাহার (starvation) হয় না এবং সব প্রসেসই CPU তে এক্সিকিউট করার সুযোগ পায়।

(স্লাইড সময় :10:39 দেখুন)

মনে করে দেখুন এই শিডিউলারটিকে আমরা $O(1)$ শিডিউলার বলে অভিহিত করেছিলাম। অর্থাৎ এটি রেডি কিউয়ে উপস্থিত প্রসেসের সংখ্যা নির্বিশেষে একটি ধ্রুবক সময় (constant time) শিডিউলার। এবার বিশ্লেষণ করে দেখা যাক কেন এটি constant time শিডিউলার। মনে করা দেখুন যে প্রসেস শিডিউলিং এখানে দুইটি ধাপে হয়। প্রথমত অন্তত একটি প্রসেস আছে এমন ন্যূনতম সংখ্যার কিউ পাওয়া এবং সেই কিউ থেকে প্রথম প্রসেসটি বেছে নেওয়া। দ্বিতীয়টি রেডি কিউ থেকে প্রথম টাস্কটি এক্সিকিউট করা। এখন প্রশ্ন হল এই দুইটি কাজ constant time এ হয় কি করে? স্পষ্টত দ্বিতীয় ধাপটি ধ্রুব সময়ে constant time এ হয়, কারণ আমরা কিউয়ের সামনের থেকে প্রসেস বেছে নিচ্ছি যা কিউয়ের দৈর্ঘ্য নির্বিশেষে নির্দিষ্ট সময়েই এটা হতে পারে। এখন প্রশ্ন হল প্রথম ধাপটি ধ্রুবক সময়ের হয় কি করে? আমরা জানি যে শিডিউলারটি 100 থেকে শুরু করে 139 পর্যন্ত priority queue গুলিকে পরীক্ষা করে অন্তত একটি প্রসেস আছে এমন ন্যূনতম সংখ্যার কিউটিকে বেছে নেয়। এই পদ্ধতিটি constant time এ হচ্ছে বলে নাও মনে হতে পারে। শিডিউলার এটি কিভাবে করে যে অন্তত একটি প্রসেস আছে এমন ন্যূনতম সংখ্যার কিউটিকে বেছে নেওয়ার কাজটি প্রসেস সংখ্যার উপর নির্ভর না করে? এটি করার জন্য লিনাক্স শিডিউলার দুইটি জিনিস করে যার প্রথমটিকে বিটম্যাপ (bitmap) বলা হয়। এর জন্য একটি 40 bit এর bitmap ব্যবহার করা হয়। এই bitmap এ খালি নয় এমন শ্রেণির জন্য 1 এবং খালি শ্রেণির জন্য 0 রাখা থাকে। দ্বিতীয়ত এখানে একটি বিশেষ নির্দেশ (instruction) থাকে যাকে find first bit set বলা হয়। Intel এ একটি bsfl instruction থাকে যা 40 bit bitmap পরীক্ষা করে ন্যূনতম 0 নয় এমন সংখ্যাটি বেছে নেয়। এই পদ্ধতিতে 0 নয় এমন সর্বনিম্ন সংখ্যার bit টি পাওয়া যায় এবং খালি নয় এমন ন্যূনতম সংখ্যার priority queue টি নির্দিষ্ট করা যায়।

(স্লাইড সময় :13:16 দেখুন)

এবার $O(1)$ শিডিউলারের অগ্রাধিকারগুলি (priority) আরো ভাল করে দেখা যাক। আমরা দেখেছি যে real time প্রসেসগুলির অগ্রাধিকার 0 থেকে 99 নেওয়া হয় আর normal প্রসেসের অগ্রাধিকার সর্বাধিক 100 ও সর্বনিম্ন 139 হতে পারে। এখন এগুলি হল normal প্রসেসের অগ্রাধিকার অর্থাৎ normal প্রসেসের অগ্রাধিকার 100 থেকে 139 যা কিছু হতে পারে। 120 অগ্রাধিকারকে base priority বলা হয় এবং কিছু বলা না থাকলে (by default) প্রসেসের অগ্রাধিকার 120 ধরে নেওয়া হয়। যেমন, একটি .out প্রোগ্রাম যখন শুরু হয় তখন তার অগ্রাধিকার 120 নেওয়া হয়। এবার nice কমান্ডের দ্বারা প্রসেসটির অগ্রাধিকার পরিবর্তন করা যায়। nice কমান্ডের দ্বারা একটি প্রসেসের অগ্রাধিকার 120 থেকে পাঁচেক অন্য কিছু করা যায়। nice কমান্ডটি এই রকম।

nice মাইনাস n ক্যাপিটাল এন ডট স্ল্যাশ এ ডট আউট যেখানে ক্যাপিটাল এন এর মান প্লাস 19 থেকে মাইনাস 20 হতে পারে। এখানে আমরা যে মানটি দিই সেটি base priority এর সাথে যোগ হয়। যেমন যদি আমরা লিখি nice মাইনাস n এবং প্লাস 19 ডট স্ল্যাশ এ ডট আউট তাহলে আমরা যে প্রসেসটি শুরু করছি তার অগ্রাধিকার 120 যোগ 19 অর্থাৎ 139 হবে। সুতরাং প্রসেসটির অগ্রাধিকার যা হতে পারে তার মধ্যে ন্যূনতম হবে। অপরদিকে যদি আমরা লিখি nice মাইনাস n এবং মাইনাস 20 ডট স্ল্যাশ এ ডট আউট তাহলে প্রসেসটির অগ্রাধিকার 100 হবে। সুতরাং প্রসেসটি normal process ধরে নিয়ে তার অগ্রাধিকার যা হতে পারে তার মধ্যে সর্বোচ্চ হবে। এগুলি হল প্রসেসগুলির স্ট্যাটিক অগ্রাধিকার (static priority) যা প্রসেসগুলি এক্সিকিউশনের প্রারম্ভে পেয়ে যায়। যায়।

(স্লাইড সময় :15:19 দেখুন)

স্ট্যাটিক বা স্থির অগ্রাধিকার (static priority) ছাড়াও, শিডিউলার ডায়নামিক বা পরিবর্তনশীল অগ্রাধিকার (dynamic priority) বলে এক প্রকার অগ্রাধিকার ধার্য করে। এই ডায়নামিক অগ্রাধিকার হিউরিস্টিক দ্বারা নির্ধারিত হয় এবং প্রসেসটি batch না interactive তার উপর নির্ভর করে। বস্তুত প্রসেসটি বেশি I/O অপারেশন করে না বেশি CPU bound তার উপর ধার্য হয়। শিডিউলার উপস্থিত কিছু হিউরিস্টিক ব্যবহার করে dynamic priority হিসেব করে। বস্তুত এটি default 120 অগ্রাধিকার মান নিয়ে বা nice এর মান আনুযায়ী অগ্রাধিকারের মান নিয়ে তার থেকে bonus বলে একটি সংখ্যা বিয়োগ করে এবং তার সাথে 5 যোগ করে। প্রথমে static priority থেকে bonus বিয়োগ করে তার সাথে 5 যোগ করা হয় এবং প্রাপ্ত মান ও 139 মধ্যে ছোট মানটি নেওয়া হয়। এবার শেষ প্রাপ্ত মান ও 100 এর মধ্যে যেটি বড় সেই মানটি dynamic priority হিসেবে নেওয়া হয়। ধরা যাক আমরা default static priority 120 নিয়ে শুরু করেছি, এবং মনে করি bonus নেওয়া হল 3, তাহলে 120 বিয়োগ 3 যোগ 5 করে হল 122। 122 এবং 139 এর মধ্যে কম হল 122। এবার 122 এবং 139 এর মধ্যে ন্যূনতম মান 122 কে নেওয়া হল, আর 122 এবং 100 এর মধ্যে বড় মানটি অর্থাৎ 122 কে dynamic priority ধার্য করা হল। এবার যেটি গুরুত্বপূর্ণ সেটি হল bonus। এই bonus এর মান কি ভাবে ধার্য করা হবে? বস্তুত bonus এর মান 0 থেকে 10 এর মধ্যে নেওয়া হয়। Bonus যদি 5 এর কম হয় তার অর্থ প্রসেসটির user interaction কম অর্থাৎ সেটি বেশি CPU bound। সুতরাং প্রসেসটির dynamic priority কমিয়ে দেওয়া হয় এবং 139 এর কাছাকাছি করা হয়। অপরদিকে যদি bonus এর মান 5 এর বেশি হয় তার অর্থ হল প্রসেসটির user interaction বেশি অর্থাৎ সেটি বেশি I/O bound। সুতরাং প্রসেসটির dynamic priority বাড়িয়ে দেওয়া হয় এবং 100 এর কাছাকাছি করা হয়। সুতরাং এই দুটি উদাহরণে দেখা গেল যে bonus 3 নিলে dynamic priority 122 হল অর্থাৎ প্রসেসটি বেশি CPU bound যার ফলে সিস্টেমে সেটির অগ্রাধিকার কম করা হল। অপরদিকে bonus 8 নিয়ে একই ভাবে হিসাব করলে দেখা যাবে dynamic priority এর মান কমে যাবে। এবার গুরুত্বপূর্ণ ব্যাপারটি হল bonus এর মান কি ভাবে ধার্য করা যাবে?

(স্লাইড সময় :18:11 দেখুন)

শিডিউলারে bonus নির্ধারণ করা হয় প্রসেসটি কতক্ষণ নিষ্ক্রিয় বা সুপ্ত (sleeping) থাকে তার উপর। এখানে অনুমান করে নেওয়া হয় যে I/O bound প্রসেস বেশি ক্ষণ সুপ্ত বা নিষ্ক্রিয় থাকে, ফলে বেশি অগ্রাধিকার পাওয়ার যোগ্য। সুতরাং I/O bound প্রসেসগুলির bonus বেশি হবে। অপরদিকে CPU bound প্রসেসগুলি কম সময় সুপ্ত থাকে তাই এদের অগ্রাধিকার কম। এই তালিকাটি থেকে দেখা যাবে যে কোন প্রসেসের গড় সুপ্তির সময় (average sleep time) 200 থেকে 300 millisecond হলে সেটিকে bonus 2 দেওয়া হয়। এটি কেবলমাত্র একটি হিউরিস্টিক যেটি কার্ণেলে ফর্মুলা হিসেবে ব্যবহার করা হয়। অপরদিকে কোন প্রসেসের গড় সুপ্তির সময় (average sleep time) যদি 800 এর সমান বা বেশি কিন্তু 900 millisecond এর কম হয় তাহলে সেটিকে বৃহত্তর bonus 8 দেওয়া হয়। এইভাবে যে প্রসেসগুলিকে বৃহত্তর bonus দেওয়া হয় যেগুলি বেশি সময় সুপ্ত থাকে এদের interactive প্রসেস ধরা হয় এবং এদের dynamic priority 100 এর কাছাকাছি হয়। অপরদিকে যে প্রসেসগুলি কম সময় সুপ্ত থাকে তাদের CPU bound প্রসেস ধরা হয়, এদের bonus কম হয় এবং এর ফলে এদের dynamic priority কমিয়ে দেওয়া হয় অর্থাৎ 139 এর দিকে করা হয়।

(স্লাইড সময় :19:40 দেখুন)

সুতরাং, এই dynamic priority কীভাবে নির্ধারিত অ্যালগরিদমকে প্রভাবিত করে? মূলত যদি আপনি এক্টিভ এবং এক্সপায়ার্ড রান কিউএর আলোচনায় ফিরে আসেন, তাহলে দেখবেন যে dynamic priority দ্বারা এটি নির্ধারণ করা হয় কোন প্রসেস কোন নির্দিষ্ট কিউয়ে স্থাপন করা হবে। উদাহরণস্বরূপ, মনে করা যাক আমরা 100 তম অগ্রাধিকার শ্রেণী থেকে একটি নির্দিষ্ট প্রসেসকে এক্সিকিউট করার জন্য নির্বাচন করলাম এবং এক্সিকিউট করার পরে এর গড় সুপ্তির সময় থেকে bonus হিসেব করে এটিকে সংশ্লিষ্ট অগ্রাধিকার শ্রেণি (priority class) তে স্থাপনা করলাম। সুতরাং গড় সুপ্তির সময়ের উপর ভিত্তি করে যখন প্রসেসটিকে expired run queue তে রাখা হবে তখন কোন শ্রেণীতে সেটিকে স্থাপন করা হবে তা সেটির dynamic priority এর উপর নির্ভর করবে।

(স্লাইড সময় :20:28 দেখুন)

তাই dynamic priority ছাড়াও শিডিউলার time slice নির্ধারণ করে। মূলত ইন্টারেক্টিভ প্রসেসগুলির উচ্চ অগ্রাধিকার থাকে, কিন্তু তারা বেশি I / O bound এবং এদের CPU বাস্ট কম হওয়ার সম্ভবনা থাকে। এই interactive প্রসেসগুলিকে বেশি time slice দেওয়া হয় যাতে এগুলির বাস্ট time শেষ হওয়ার পূর্বে এদের CPU থেকে বেদখল (pre-empt) করা হয় না। সুতরাং, time slice নির্দিষ্ট করার জন্য এটির মত আমরা আরো heuristics ব্যবহার করি। মূলত যদি অগ্রাধিকার 120 এর চেয়ে কম হয় তবে এটি একটি ইন্টারেক্টিভ প্রসেস, এর time slice নির্ণয় করতে 140 থেকে অগ্রাধিকার বিয়োগ করে 20 মিলিসেকেন্ড দিয়ে গুণ করা হয়। তবে যদি

এটি একটি ব্যাচ প্রসেস হয়, যার CPU এ সময় বেশি লাগে, এর time slice নির্ণয় করতে 140 থেকে অগ্রাধিকার বিয়োগ করে 5 মিলিসেকেন্ড দিয়ে গুণ করা হয়। ফলস্বরূপ এই তালিকা থেকে দেখা যায় যে nice মান মাইনাস 20 নিলে static priority 100 পাওয়া যায়। সুতরাং, এটি একটি ইন্টারেক্টিভ প্রসেস এবং একে 800 মিলিসেকেন্ডের বড় time slice দেওয়া হয়। অপরদিকে যে প্রসেসটির static priority 139 অর্থাৎ 120 এর বেশি আর nice ভ্যালু 19 সেটিকে সর্বনিম্ন time slice 5 মিলিসেকেন্ড দেওয়া হয়। সুতরাং, আমরা দেখতে পাচ্ছি যে অগ্রাধিকার অনুযায়ী একটি প্রসেসকে যে time slice দেওয়া হয় তা নির্দিষ্ট।

(স্লাইড সময় :22:15 দেখুন)

সুতরাং সংক্ষেপে বলা যাতে পারে যে $O(1)$ শিডিউলার হল একটি multi level feedback কিউ যার 40 টি অগ্রাধিকার শ্রেণী (priority class) আছে। প্রাথমিক অগ্রাধিকার 120 হলেও সেটি nice কমান্ড দ্বারা পরিবর্তন করা যায়। Heuristic দ্বারা নির্ধারিত dynamic priority গড় সুপ্তির সময় (average sleep time) এর উপর নির্ভর করে। প্রতিটি প্রসেসের time slice তার dynamic priority এর উপর নির্ভর করে।

(স্লাইড সময় :22:39 দেখুন)

$O(1)$ শিডিউলারের সীমাবদ্ধতাগুলি নিম্নরূপ: $O(1)$ শিডিউলার অনেক জটিল heuristics ব্যবহার করে interactive এবং non-interactive প্রসেসের মধ্যে পার্থক্য করে। সুতরাং, যেমন time slice এবং অগ্রাধিকারের মধ্যে একটি সম্পর্ক আছে, তেমনই অগ্রাধিকার এবং time slice এক রকম হয় না। 100 এবং 139 অগ্রাধিকারের প্রসেসগুলির time slice দেখলে দেখা যাবে যে দুইটির পাল্লা (range) এক রকম নয়। তালিকাটি পুনরায় দেখলে দেখা যাবে যে 100 থেকে 110 অগ্রাধিকারের তফাতে time slice পরিবর্তিত হয়েছে 200 মিলিসেকেন্ড, যেখানে 130 থেকে 139 অগ্রাধিকারের তফাতে time slice পরিবর্তিত হয়েছে মাত্র 45 মিলিসেকেন্ড। এই সঙ্গে আমরা এই নির্দিষ্ট ভিডিও বক্তৃতা শেষ করব। পরবর্তী বক্তৃতায় আমরা CFS শিডিউলার নিয়ে আলোচনা করব, যা বর্তমান বা সর্বাধুনিক লিনাক্স কার্নেলে ব্যবহার করা হয়। ধন্যবাদ।